TARKA

v0.5 — Experimental Interpreter

Syntax Rules & Quick Reference

This guide describes the syntax implemented in the current Tarka v0.5 interpreter. It is based on the current interpreter source, not a future roadmap.

Language detail	Tarka v0.5 rule
Source files	UTF-8 text; the interpreter can run a .tarka file by passing its filename.
Statement ending	Normally a newline. A statement may also end immediately before a closing brace or end of file.
Blocks	Curly braces: { and }.
Comments	Single-line comments begin with //.
Identifiers	Start with a letter or underscore; may contain letters, digits and underscores.

Important: Tarka v0.5 is experimental. Syntax and behaviour may change in later versions.

1. Keywords

The following words are reserved by the v0.5 lexer and cannot be used as ordinary variable or function names.

Keyword	Meaning / use
mana	Declare a variable
mudraya	Print a value
pravesha	Read input into a variable
yadi	If condition
anyatha	Else branch
yavat	While loop
avartta	For loop
kriya	Declare a function
pratyarpanam	Return from a function
virama	Break out of a loop
anuvartta	Continue to the next loop iteration
satyam	Boolean true
asatyam	Boolean false
shunyam	Null value
cha	Logical AND
va	Logical OR
na	Logical NOT

2. Basic program structure

A Tarka program is a sequence of statements. Statements are usually separated by newlines. Blocks contain statements inside curly braces.

```
// This is a comment
mana language = "Tarka"
mudraya language
```

Whitespace spaces and tabs are ignored between tokens. Newlines separate statements outside expressions and block headers.

3. Variables and assignment

Declaration

```
mana name = "Tarka"
mana age = 21
mana price = 99.5
mana active = satyam
mana result = shunyam
```

Use **mana** to declare a variable. A value is required at declaration time.

Reassignment

```
age = 22
```

```
name = "Tarka Language"
```

Reassignment uses = without the mana keyword. The variable must already exist.

4. Data types and literals

Type	Example
Integer	42
Float	3.14
String	"Namaste"
Boolean	satyam / asatyam
Null	shunyam
List	[10, 20, 30]
Dictionary	{"name": "Tarka", "age": 21}

Numbers without a decimal point are integers. Numbers with a decimal point followed by digits are floats. Strings use double quotes.

String escape sequences

```
"First line\nSecond line"
"Column1\tColumn2"
"She said: \"Hello\""
"Backslash: \\"
```

Supported escapes are **\n**, **\t**, **\"** and ****. Unknown escapes and strings that reach a newline or end of file before the closing quote produce a lexer error.

5. Operators

Category	Operators / keywords	Example
Arithmetic	+ - * / %	total = 10 + 5 * 2
Comparison	== != < <= > >=	age >= 18
Logical AND	cha	age >= 18 cha active
Logical OR	va	is_member va is_admin
Logical NOT	na (also !)	na active
Grouping	( )	(a + b) * 2

Division uses / and returns a floating-point result. Logical operators require Boolean operands. Conditions in if and loop headers must evaluate to a Boolean.

Operator precedence, from higher to lower: unary NOT / minus; multiplication, division and modulo; addition and subtraction; comparisons; equality; AND; OR; assignment.

6. Output and input

Print

```
mudraya "Hello, Tarka!"
mana score = 95
mudraya score
```

Read input

```
pravesha city
mudraya "Welcome from:"
mudraya city
```

pravesha is followed by a variable name. The entered value is read as a string.

7. Conditions

```
mana age = 20

yadi (age >= 18) {
    mudraya "Adult"
} anyatha {
    mudraya "Minor"
}
```

The condition must be enclosed in parentheses. Each branch is a brace-delimited block. The anyatha branch is optional.

8. Loops

While loop

```
mana count = 1

yavat (count <= 3) {
    mudraya count
    count = count + 1
}
```

A yavat loop repeats while its Boolean condition is true.

For loop

```
avartta (mana i = 1; i <= 5; i = i + 1) {
    mudraya i
}
```

The for header has three parts separated by semicolons: initializer, condition and increment. The initializer may declare a variable with mana or be an expression. The condition and increment may be omitted by the parser.

Break and continue

```
avartta (mana i = 1; i <= 5; i = i + 1) {
    yadi (i == 3) {
        anuvartta
    }
    yadi (i == 5) {
        virama
    }
    mudraya i
}
```

virama exits the current loop. **anuvartta** skips the rest of the current iteration and continues the loop. Both must be used inside a loop.

9. Functions

```
kriya add(a, b) {
    pratyarpanam a + b
}

mana answer = add(10, 20)
mudraya answer
```

Declare a function with kriya, followed by its name, comma-separated parameters in parentheses, and a block. Call it using its name and arguments in parentheses.

Return values and recursion

```
kriya factorial(n) {
    yadi (n <= 1) {
        pratyarpanam 1
    }
    pratyarpanam n * factorial(n - 1)
}

mudraya factorial(5)
```

pratyarpanam returns a value. A function that reaches its end without a return expression returns shunyam. Recursion is supported.

10. Lists and dictionaries

Lists

```
mana numbers = [10, 20, 30]
mudraya numbers[0]
numbers[1] = 25
append(numbers, 40)
mudraya len(numbers)
```

List indexes start at 0. Indexing uses square brackets. An indexed list element can be reassigned.

Dictionaries

```
mana person = {
    "name": "Tarka",
    "age": 21
}

mudraya person["name"]
person["age"] = 22
mudraya person
```

Dictionary entries use key: value pairs separated by commas. Keys and values are expressions. Access and update entries with square brackets.

Lists and dictionaries may contain nested values. A trailing comma before the closing bracket or brace is accepted.

11. Built-in functions

Built-in	Purpose
len(value)	Return the length of a string, list or dictionary.
str(value)	Convert a value to its Tarka display string.
int(value)	Convert a number or string to an integer.
float(value)	Convert a number or string to a float.
type(value)	Return a type name such as Integer, String, List or Dictionary.
append(list, value)	Append a value to a list.
remove(list, value)	Remove the first matching value from a list.
keys(dictionary)	Return a list of dictionary keys.
values(dictionary)	Return a list of dictionary values.
range(start[, stop[, step]])	Create a list of integers using Python-style range arguments.

range accepts one, two or three integer arguments. For example, range(1, 5) produces [1, 2, 3, 4].

12. Comments, errors and current limitations

```
// Tarka ignores the rest of this line
mana total = 10 + 20
mudraya total
```

The interpreter reports lexer, parser and runtime errors with a Tarka Error message. Examples include unexpected characters, missing expressions, undefined variables, division by zero, invalid indexing and incorrect function argument counts.

Not part of the current v0.5 syntax: imports/modules, classes/objects, try/catch-style error handling, user-defined types, and a larger standard library. These are future development possibilities, not supported syntax in this version.

13. Complete example

```
// A small Tarka program

mana name = "Tarka"
mana numbers = [2, 4, 6, 8]

kriya sum_list(items) {
    mana total = 0
    avartta (mana i = 0; i < len(items); i = i + 1) {
        total = total + items[i]
    }
    pratyarpanam total
}

yadi (len(numbers) > 0) {
    mudraya "Hello, " + name
    mudraya sum_list(numbers)
} anyatha {
    mudraya "The list is empty"
}
```

Version note: This reference is for the Tarka v0.5 interpreter source currently provided. Verify examples against the exact interpreter build you publish, and update this document when syntax changes.